

Multi-Source Candidate Data Transformer

Design Document

Jashanpreet Kaur | jashan100106@gmail.com | Eightfold Engineering Intern Assignment

What this project does

This project takes information about a job candidate from several different places (a recruiter's spreadsheet, an applicant tracking system, a resume file, and a public GitHub profile), cleans it up, and combines it into one single, trustworthy candidate profile. Along the way, it keeps track of which source each piece of information came from and how confident it is in each value.

How the data flows through the system

The system works in clear stages, one after another. First, each source is read by its own dedicated piece of code, so a problem reading a resume can never affect how a spreadsheet is read. Next, every value is cleaned up into a standard format, for example, phone numbers are converted into one consistent international format, and different spellings of the same skill (such as "ReactJS" and "react.js") are combined into one name.

After that, the cleaned values from all sources are combined into a single profile. When two sources disagree on something like a person's name or years of experience, the system picks the more trustworthy source rather than guessing. When a value can reasonably have more than one entry, such as a list of emails or phone numbers, the system keeps every distinct value instead of throwing any away. The system then calculates a confidence score for each field, checks that everything makes sense, and finally formats the result into the exact output format that was requested.

How conflicting information is resolved

Each source is ranked by how trustworthy it normally is. Information entered directly by an employer or recruiter is trusted the most, followed by information from a resume, followed by information from a public GitHub profile, since a GitHub display name is often just a nickname someone chose for themselves. When two sources disagree on a single-value field, the system uses this ranking to decide which value to keep. For fields that can have multiple valid values, like emails, phone numbers, or skills, the system keeps all of them rather than picking only one.

Every field's confidence score reflects how trustworthy its source was, whether the value could be cleanly understood and standardized, how many different sources agreed on it, and whether a conflict had to be resolved. An overall confidence score for the whole profile is the average of all of these individual scores.

Customizing the output

The system can produce its results in more than one format without needing any code changes. A configuration file controls which fields are included in the final result, what each field should be called, how certain values like phone numbers or country names should be standardized, and what should happen if a requested field turns out to be missing, whether that means leaving it blank, leaving it out entirely, or stopping with a clear error. After the output is generated, the system checks it one more time to make sure it actually matches what was requested.

Examples of tricky situations this system handles

- Sources report different phone numbers, or slightly different spellings of the same name; the system resolves these sensibly instead of failing or guessing.
- A phone number is missing its country code; the system assumes one but lowers its confidence in that value rather than treating it as fully certain.
- An input file is broken, badly formatted, or missing entirely; the system skips that one source and still produces a complete result from whatever is available, instead of crashing.
- An external service such as GitHub is temporarily unavailable or rate-limited; the system notices quickly and moves on, rather than getting stuck waiting.
- A field marked required in the requested output ends up missing; the system raises a clear error instead of silently returning incomplete data.

What was intentionally left out, and why

A few things were deliberately not built, given the time available, and are noted here rather than left unexplained. Reading data from LinkedIn was left out, since there is no proper, permitted way to access it automatically; the only option would have meant scraping the site, which goes against LinkedIn's own rules and would be unreliable to depend on. Reading free-text recruiter notes was also left out; the system is already designed to support this as a future addition, but the actual parsing was not built, so more time could go into fully finishing and testing the other four sources. The system also does not automatically guess which input files belong to the same candidate; that is specified directly instead, since auto-matching is a much harder problem on its own and was outside what this assignment actually tests.